mTLS Rotation Failure Prediction – Implementation Plan & Dataset Setup

Project: Predictive Certificate Lifecycle Management in Service Mesh Architectures (Istio + Chaos Mesh + ML)

Purpose of this doc: what to implement, in order, with tool verification first and a step-by-step method to build our own labeled dataset.

1. What we are implementing (big picture)

There is no public dataset for mTLS certificate rotation failures, so building one is itself part of the contribution. The implementation has six phases. Today's review should focus on Phases 0–2 (environment, cluster, observability) and the dataset design in Phases 3–5.

Phase	Goal	Output
0	Verify tools and machine resources	Passing checklist script
1	Kind cluster + Istio with short-lived certificates	Running mesh, rotations every ~30 min
2	Observability (Prometheus, istiod and Envoy metrics, logs)	Metrics queryable via PromQL
3	Workload and traffic generator	Steady baseline load
4	Chaos Mesh fault injection scenarios + ground-truth event logging	Labeled failure events
5	Collector, feature engineering, labeling, dataset export	Versioned Parquet/CSV dataset + dataset card
6 (later)	XGBoost model, FastAPI inference, CronJob retraining, remediation	Prediction service

Three-tier target architecture (for later phases): data collection → nightly retraining (Kubernetes CronJob) → inference (FastAPI). Automated remediation (e.g., scale istiod replicas on high-risk prediction) is the practical core; a custom CSR approval controller is a stretch item.

2. Phase 0 – Verify required tools and software

2.1 Required tools

Tool	Why we need it	Minimum
Docker (daemon running)	Kind nodes run as containers	24+
kind	Local multi-node Kubernetes cluster	0.20+
kubectl	Cluster control	within one minor version of the cluster
Helm	Install Chaos Mesh, kube-state-metrics	3.12+
istioctl	Install and inspect Istio, read proxy certificates	1.22+ (pin one version for the whole study)
Python	Collector, feature pipeline, model	3.10+
pip / venv	Python dependencies	current
git	Version control for configs and dataset scripts	any
jq	Parse JSON from kubectl / istioctl	any
openssl	Inspect certificate validity dates	any
curl	Smoke tests	any

Python packages: requests, pandas, numpy, pyarrow, scikit-learn, xgboost, pyyaml, matplotlib, jupyter. Later: fastapi, uvicorn, joblib.

Machine resources (recommended): 4+ CPU cores, 8 GB RAM free (16 GB is comfortable), 30 GB free disk. If Docker is on Windows, use WSL2 with Docker Desktop integration enabled.

2.2 Verification script

Save as `scripts/00_verify_env.sh` and run `bash scripts/00_verify_env.sh`.

```
#!/usr/bin/env bash
set -u
PASS=0; FAIL=0

check() {  # name, version-command
  if command -v "$1" >/dev/null 2>&1; then
    v=$(eval "$2" 2>&1 | head -n1)
    printf "[ OK ] %-10s %s\n" "$1" "$v"; PASS=$((PASS+1))
  else
    printf "[MISS] %-10s not installed\n" "$1"; FAIL=$((FAIL+1))
  fi
}

check docker    "docker --version"
check kind      "kind version"
check kubectl   "kubectl version --client"
check helm      "helm version --short"
check istioctl  "istioctl version --remote=false"
check python3   "python3 --version"
check pip3      "pip3 --version"
check git       "git --version"
check jq        "jq --version"
check openssl   "openssl version"
check curl      "curl --version"

echo "--- Docker daemon ---"
if docker info >/dev/null 2>&1; then echo "[ OK ] daemon running"; else echo
"[FAIL] docker daemon not reachable"; FAIL=$((FAIL+1)); fi

echo "--- Resources ---"
echo "CPU cores: $(nproc 2>/dev/null || sysctl -n hw.ncpu)"
(free -h 2>/dev/null | awk '/Mem:/ {print "RAM total/free: " $2 " / " $7}')
|| true
df -h . | awk 'NR==2 {print "Disk free here: " $4}'

echo "--- Python packages ---"
python3 - <<'EOF'
import importlib
for m in ["requests","pandas","numpy","pyarrow","sklearn","xgboost","yaml"]:
    try:
        mod = importlib.import_module(m)
        print(f"[ OK ] {m} {getattr(mod,'__version__','')}")
    except Exception:
```

```
        print(f"[MISS] {m}")
EOF

echo "Summary: $PASS tools OK, $FAIL problems"
```

2.3 Install anything missing (Linux / WSL2 example)

```
# kind
curl -Lo ./kind https://kind.sigs.k8s.io/dl/v0.23.0/kind-linux-amd64 && chmod
+x kind && sudo mv kind /usr/local/bin/
# kubectl: follow https://kubernetes.io/docs/tasks/tools/
# helm
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 |
bash
# istioctl (pin the version)
curl -L https://istio.io/downloadIstio | ISTIO_VERSION=1.22.3 sh -
export PATH=$PWD/istio-1.22.3/bin:$PATH
# python env
python3 -m venv .venv && source .venv/bin/activate
pip install requests pandas numpy pyarrow scikit-learn xgboost pyyaml
matplotlib jupyter
```

Re-run the verification script until every line shows OK. Record the exact versions in the paper's experimental setup section.

2.4 Suggested repo layout

```
mtls-predict/
  scripts/            # 00_verify_env.sh, 01_cluster.sh, run_experiment.py
...
  cluster/            # kind-config.yaml, istio-config.yaml
  workload/           # app manifests, load generator
  chaos/              # Chaos Mesh scenario YAMLs
  collector/          # prometheus + log collectors
  data/raw/           # per-run raw metrics and logs
  data/processed/     # feature tables (parquet)
  notebooks/
  docs/               # dataset card, experiment log
```

3. Phase 1 – Cluster and Istio with short-lived certificates

Default Istio workload certificates live 24 hours and rotate at about 50% of lifetime, so natural rotations are too rare to build a dataset. We shorten the TTL so each pod rotates many times per experiment.

3.1 Create the Kind cluster

`cluster/kind-config.yaml`:

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
  - role: control-plane
  - role: worker
  - role: worker
  - role: worker
```

```
kind create cluster --name mtls-lab --config cluster/kind-config.yaml
kubectl get nodes          # expect 4 nodes Ready
```

3.2 Install Istio with short TTL

`cluster/istio-config.yaml`:

```
apiVersion: install.istio.io/v1alpha1
kind: IstioOperator
spec:
  profile: demo
  meshConfig:
    defaultConfig:
      proxyMetadata:
        SECRET_TTL: "1h"           # requested workload cert lifetime
  components:
    pilot:
      k8s:
        env:
          - name: DEFAULT_WORKLOAD_CERT_TTL
            value: "1h"
```

```
istioctl install -y -f cluster/istio-config.yaml
kubectl -n istio-system get pods
kubectl label namespace default istio-injection=enabled --overwrite
```

Note: confirm the exact env var names and the minimum allowed TTL against the documentation of the Istio version you pinned. If a 1h TTL is rejected, use 2h–3h. Rotation is expected at roughly half the lifetime (a 1h cert should renew near the 30 min mark).

3.3 Verify rotation is really happening

```
POD=$(kubectl get pod -l app=productpage -o
jsonpath='{.items[0].metadata.name}')
istioctl proxy-config secret $POD -o json \
 | jq -r '.dynamicActiveSecrets[] | select(.name=="default") |
.secret.tlsCertificate.certificateChain.inlineBytes' \
 | base64 -d | openssl x509 -noout -dates
```

Run it twice 35 minutes apart: `notBefore` should change. This is the baseline proof that the mesh rotates certificates on the shortened schedule.

4. Phase 2 – Observability

Why both metrics and logs: Prometheus gives continuous numeric time series and control-plane signals (istiod CPU/memory, CSR activity) that sidecar logs do not contain; logs give discrete error events and the ground-truth labels.

4.1 Prometheus

```
kubectl apply -f istio-1.22.3/samples/addons/prometheus.yaml
kubectl -n istio-system port-forward svc/prometheus 9090:9090 &
curl -s 'localhost:9090/api/v1/query?query=up' | jq '.status'
```

Also install kube-state-metrics for pod and node restart counts:

```
helm repo add prometheus-community https://prometheus-
community.github.io/helm-charts
helm install ksm prometheus-community/kube-state-metrics -n istio-system
```

(If it is not scraped automatically, add its service to the Prometheus scrape config.)

4.2 Candidate feature signals to confirm in the Prometheus UI

Before coding the collector, open `localhost:9090` and confirm which of these exist in your Istio version (names vary by version, so check the metric explorer):

Group	Signal (check exact name)	Meaning
istiod CA	`citadel_server_csr_count`, `citadel_server_success_cert_issuance_count`, `citadel_server_authentication_failure_count`, `citadel_server_csr_parsing_err_count`	CSR volume, issuance success, auth failures

Group	Signal (check exact name)	Meaning
istiod load	`process_cpu_seconds_total`, `container_memory_working_set_bytes` for istiod, `pilot_xds_pushes`, `pilot_proxy_convergence_time`	Control-plane stress
istio-agent / Envoy	`istio_agent_cert_expiry_seconds` (if present), Envoy `ssl` handshake and connection-failure counters	Time left on cert, TLS errors
Traffic	`istio_requests_total` by `response_code` and `response_flags`	503s and UF/URX flags during failures
Cluster	`kube_pod_container_status_restarts_total`, node ready status	Restart frequency
Rotation history	derived: time since last `notBefore` change per pod	Rotation cadence

Write the confirmed list into `docs/feature_catalog.md`. Anything not exposed by default (e.g., per-pod cert expiry) can be collected by a small script that runs the `istioctl proxy-config secret` command from Section 3.3 every 15 seconds and exports the remaining lifetime.

4.3 Logs

Collect `istio-proxy` container logs for all workload pods and the `istiod` logs for the whole run. Search for text such as certificate generation failure, CSR errors, SDS errors, and expired certificate TLS errors. Save these with `kubectl logs --since-time` into `data/raw/<run_id>/logs/`. (Loki is optional; plain files are fine for a first dataset.)

5. Phase 3 – Workload and traffic

Use Istio's Bookinfo sample (small, well known, multi-service) and a steady load generator.

```
kubectl apply -f istio-1.22.3/samples/bookinfo/platform/kube/bookinfo.yaml
kubectl apply -f istio-1.22.3/samples/bookinfo/networking/bookinfo-
gateway.yaml
kubectl get pods     # each pod should be 2/2 (app + sidecar)
```

Load generator (run as an in-cluster pod so it is part of the mesh):

```
kubectl run loadgen --image=fortio/fortio --restart=Never -- \
  load -qps 20 -t 0 -c 8 http://productpage:9080/productpage
```

Run a 1–2 hour no-fault baseline first. This gives the "healthy" class and the normal rotation pattern. Optional second workload (Online Boutique) later for generalization.

6. Phase 4 – Fault injection and ground-truth events

6.1 Install Chaos Mesh (Kind uses containerd)

```
helm repo add chaos-mesh https://charts.chaos-mesh.org
kubectl create ns chaos-mesh
helm install chaos-mesh chaos-mesh/chaos-mesh -n chaos-mesh \
  --set chaosDaemon.runtime=containerd \
  --set chaosDaemon.socketPath=/run/containerd/containerd.sock
kubectl -n chaos-mesh get pods     # controller, daemon, dashboard Running
```

6.2 Fault scenario catalog

Each scenario targets a plausible cause of a rotation failure. Keep intensity and duration randomized across runs so the model does not memorize one pattern.

ID	Scenario	How	Why it causes rotation trouble
S0	Baseline	No fault	Healthy class
S1	istiod CPU stress	StressChaos on `app=istiod`	Slow CSR signing
S2	istiod memory pressure	StressChaos memory	Control-plane instability
S3	istiod pod kill / restart loop	PodChaos `pod-kill`	CA unavailable during renewal window
S4	Network loss/delay workload↔istiod	NetworkChaos on traffic to istiod:15012	Sidecar cannot fetch renewed cert
S5	Network partition (hard)	NetworkChaos `partition`	Certificate actually expires
S6	Clock skew	TimeChaos on workload pods	Premature or missed expiry
S7	Node restart	`docker restart mtls-lab-worker2`	Mass re-issuance, CSR burst
S8	CSR burst	Rapid scale up/down of deployments	Queue pressure on CA

ID	Scenario	How	Why it causes rotation trouble
S9	Missing ServiceAccount	Delete SA, restart pods (our Techmojo incident)	Identity cannot be issued
S10	Combined	S1 + S4 together	Realistic compound failure

Example (S1) `chaos/s1-istiod-cpu.yaml`:

```
apiVersion: chaos-mesh.org/v1alpha1
kind: StressChaos
metadata:
  name: istiod-cpu-stress
  namespace: istio-system
spec:
  mode: one
  selector:
    namespaces: [istio-system]
    labelSelectors: {app: istiod}
  stressors:
    cpu: {workers: 2, load: 90}
  duration: "10m"
```

Example (S4) `chaos/s4-net-loss.yaml`:

```
apiVersion: chaos-mesh.org/v1alpha1
kind: NetworkChaos
metadata:
  name: workload-to-istiod-loss
  namespace: default
spec:
  action: loss
  mode: all
  selector:
    namespaces: [default]
  direction: to
  target:
    mode: all
    selector:
      namespaces: [istio-system]
      labelSelectors: {app: istiod}
  loss: {loss: "60", correlation: "25"}
  duration: "10m"
```

S9 helper:

```
kubectl delete sa bookinfo-reviews
kubectl rollout restart deploy/reviews-v1     # pod creation/identity fails
# restore afterwards
kubectl create sa bookinfo-reviews
```

6.3 Experiment runner (the key script)

`scripts/run_experiment.py` should do the following for every run, so that timing is exact:

1. Generate a `run_id` and pick a scenario, intensity, and duration (seeded random, store the seed).
2. Record `t_start` and confirm baseline traffic is healthy for 10 minutes.
3. Apply the chaos YAML; record `t_fault_start`.
4. Keep collecting; remove the chaos at `t_fault_end`; record it.
5. Keep collecting for a 15 minute recovery period.
6. Write one row to `data/raw/experiment_log.csv`: `run_id, scenario, intensity, seed, t_start, t_fault_start, t_fault_end, t_end`.
7. Clean up (delete chaos objects, restore SA, wait for pods Ready) before the next run.

Repeat each scenario at least 15–20 times plus several long baseline runs.

6.4 Defining the ground-truth failure event

This definition is the heart of the labeling and must be stated precisely in the paper. Proposed definition: a rotation failure occurs for a workload at time T when any of these is observed:

- the workload certificate passes its `notAfter` without being replaced (detected by the 15 second cert-expiry poller), or
- the istio-proxy log shows a certificate fetch or SDS generation error that persists beyond the grace period, or
- Envoy reports TLS handshake failures / 503 spikes attributable to an expired or missing certificate.

Because TTLs are short, we also want near-miss events (rotation succeeded but late). Record rotation latency (time from the planned renewal point to the new `notBefore`) as an extra column; it supports both labeling and the analysis section.

Write an `event_log.csv` with: `run_id, workload, event_type, event_time, evidence_source`.

7. Phase 5 – Collection, features, labels, and dataset export

7.1 Collector

`collector/prom_collector.py` scrapes the Prometheus HTTP API on a fixed step (15 s) for every PromQL expression in the feature catalog and writes one tidy file per run.

```
import requests, pandas as pd, time

PROM = "http://localhost:9090"
QUERIES = {
  "istiod_cpu": 'sum(rate(container_cpu_usage_seconds_total{namespace="istio-
system",pod=~"istiod.*"}[1m]))',
  "istiod_mem": 'sum(container_memory_working_set_bytes{namespace="istio-
system",pod=~"istiod.*"})',
  "csr_rate":   'sum(rate(citadel_server_csr_count[1m]))',
  "issue_ok":   'sum(rate(citadel_server_success_cert_issuance_count[1m]))',
  "restarts":
'sum(increase(kube_pod_container_status_restarts_total[5m]))',
  "req_5xx":    'sum(rate(istio_requests_total{response_code=~"5.."}[1m]))',
}

def fetch(q, start, end, step="15s"):
    r = requests.get(f"{PROM}/api/v1/query_range",
                     params={"query": q, "start": start, "end": end, "step":
step}, timeout=30)
    r.raise_for_status()
    res = r.json()["data"]["result"]
    if not res: return pd.Series(dtype=float)
    vals = res[0]["values"]
    return pd.Series({pd.to_datetime(t, unit="s"): float(v) for t, v in
vals})

def collect(run_id, start, end):
    df = pd.DataFrame({k: fetch(q, start, end) for k, q in QUERIES.items()})
    df.index.name = "ts"
    df.to_parquet(f"data/raw/{run_id}/metrics.parquet")
    return df
```

Also store the per-pod cert-expiry series from the poller (`seconds_to_expiry`, `last_rotation_age`) into the same run folder.

7.2 Feature engineering

For each 15 s timestamp and each workload (or each mesh-level row), build:

- raw signals from Section 4.2,
- rolling mean / max / std over 1, 5, and 15 minute windows,
- rate of change (slope) of istiod CPU, memory, and CSR rate,
- CSR success ratio = issuance success / CSR count,
- `seconds_to_expiry`, `time_since_last_rotation`, rotation latency history (last 3),
- node/pod restart count in the last 5 and 15 minutes,
- `scenario` is never a feature (it would leak the answer).

7.3 Labeling for prediction

Choose a prediction horizon H (start with 10 minutes, also try 5 and 15). For each row at time t:

`label(t) = 1` if a ground-truth rotation failure event (Section 6.4) occurs in `(t, t + H]`, else `0`.

Exclude rows that fall inside an already-failing period (the failure has already happened, so predicting it is meaningless) or mark them with a separate `already_failing` flag.

7.4 Train / validation / test split

Split by experiment run, never by random row. Adjacent rows within one run are nearly identical, and a random split leaks information and gives falsely high scores. Suggested: 70% of runs train, 15% validation, 15% test, stratified so every scenario appears in each split. Also hold out one whole scenario (e.g., S6) for a generalization test.

7.5 Class imbalance

Expect far fewer positive rows than negative ones. Handle with `scale_pos_weight` in XGBoost and report precision, recall, F1, PR-AUC, and lead time (how many minutes before the failure the alert fires) rather than accuracy.

7.6 Dataset export and documentation

- Final tables: `data/processed/features_v1.parquet` (and a CSV copy for sharing).
- Columns: `run_id, ts, workload, <features...>, label_H5, label_H10, label_H15, already_failing`.
- Record the dataset card in `docs/dataset_card.md`: Istio / Kubernetes / Chaos Mesh versions, cert TTL settings, scenario list with parameters, number of runs per scenario, row counts, positive rate, collection interval, label definition, known limitations (single local cluster, synthetic faults).
- Version the data (`features_v1`, `v2`) and keep seeds so every run is reproducible.

7.7 Dataset size targets for the first version

Item	Target
Baseline (S0) runs	5 runs × 1–2 h
Fault runs per scenario (S1–S10)	15–20
Run length	about 45 min (10 pre, 10–15 fault, 15 recovery)
Total elapsed collection time	roughly 2–3 days of unattended runs
Minimum positive samples	a few hundred labeled positive rows per horizon

Start with a pilot: 2 scenarios (S1, S4) × 3 runs plus 1 baseline, run end to end to validate the pipeline before scaling up.

8. Suggested order of work (checklist)

- [ ] Run `00_verify_env.sh` and fix any MISS lines
- [ ] Create Kind cluster and install Istio with short TTL
- [ ] Prove a certificate rotation by comparing `notBefore` over time
- [ ] Install Prometheus and kube-state-metrics, confirm metric names, write `feature_catalog.md`
- [ ] Deploy Bookinfo and the load generator, run a baseline
- [ ] Install Chaos Mesh and test one fault (S1) manually
- [ ] Build the cert-expiry poller and the event log
- [ ] Build `run_experiment.py` and the collector
- [ ] Pilot run (S1, S4, baseline), build features and labels, inspect plots
- [ ] Scale to the full scenario catalog, freeze `features_v1`
- [ ] Then Phase 6: XGBoost, FastAPI, CronJob retraining, remediation

9. For today's implementation review

Suggested talking points:

- Show the verification script output and the running cluster (`kubectl get nodes`, `istioctl proxy-status`).
- Show a certificate rotation proof and a Prometheus graph of istiod metrics.
- Present the scenario catalog and the precise ground-truth failure definition.
- Explain the leakage-safe split (by run) and the horizon-based labeling.

- State clearly that the dataset generation method is itself a contribution, since no public dataset exists.

Risks and open points to mention:

- Exact metric names and TTL minimums depend on the Istio version, so the version is pinned and documented.
- Synthetic faults on a local Kind cluster may differ from production; discuss validation against the real Techmojo-style incident (S9) as the realism anchor.
- Local laptop resources can limit stress experiments; keep durations and intensities modest.